

Machine Learning-Based Anomaly Detection for Cybersecurity in Industrial IoT Environments

Evaluation of Autoencoder, Isolation Forest, and Extreme Learning Machine Approaches Using the TON-IoT Dataset

Arushi Sharma

Master's Thesis

MEng in Machine Intelligence and Data Science

2026

Master's Thesis

Arushi Sharma

Machine Learning-Based Anomaly Detection for Cybersecurity in Industrial IoT Environments. Evaluation of Autoencoder, Isolation Forest, and Extreme Learning Machine Approaches Using the TON-IoT Dataset.

Arcada University of Applied Sciences: MEng in Machine Intelligence and Data Science, 2026.

Identification number:

048236069

Commissioned by:

not commissioned

Abstract:

This thesis investigates machine learning-based anomaly detection techniques for cybersecurity in Industrial Internet of Things (IIoT) environments using the TON-IoT dataset. The study focuses on detecting Distributed Denial of Service (DDoS) attacks using Zeek network traffic logs derived from the TON-IoT dataset.

The research implements and compares three models: Isolation Forest, Autoencoder, and Extreme Learning Machine (ELM). The preprocessing pipeline includes feature engineering, one-hot encoding of categorical variables, normalization of numerical features, and transformation of raw network traffic logs into structured machine learning features suitable for anomaly detection.

The Autoencoder model is trained exclusively on normal network traffic and identifies anomalies based on reconstruction error. Isolation Forest is used as an unsuper-

vised baseline method, while ELM is evaluated as a supervised classification model using labeled data.

The experimental results demonstrate that the Autoencoder achieves strong discriminative capability, particularly when threshold selection is optimized. Isolation Forest successfully identifies statistically distinct anomalies, while ELM achieves very high classification performance under supervised conditions.

The findings highlight the importance of feature engineering, threshold selection, and model choice in anomaly detection systems. The study also demonstrates that unsupervised approaches are particularly valuable in IIoT environments where labeled attack data is limited or unavailable.

Overall, the results contribute to the development of scalable and adaptive cybersecurity solutions for Industrial IoT systems.

Keywords: IIoT, Cybersecurity, Anomaly Detection, Autoencoder, Isolation Forest, Extreme Learning Machine, TON-IoT

Contents

1 Introduction . . . 10

1.1 Background . . . 10
1.2 Problem Statement . . . 10
1.3 Research Objectives . . . 10
1.4 Research Questions . . . 11
1.5 Thesis Contributions . . . 11
1.6 Thesis Structure . . . 11

2 Related Work . . . 13

2.1 Cybersecurity in Industrial IoT Systems . . . 13
2.2 Intrusion Detection Systems (IDS) . . . 13
2.3 Anomaly Detection in Network Traffic . . . 14
2.4 Deep Learning for Anomaly Detection . . . 15
2.5 Extreme Learning Machines . . . 15
2.6 Theoretical Advantages of ELM . . . 16
2.7 Comparison of Supervised and Unsupervised Methods . . . 16
2.8 Datasets for Cybersecurity Research . . . 17
2.9 Research Gap . . . 17
2.10 Evolution of Network Intrusion Detection . . . 18
2.10.1 First Generation: Signature-Based Systems . . . 18
2.10.2 Second Generation: Statistical Anomaly Detection . . . 19
2.10.3 Third Generation: Machine Learning Integration . . . 19
2.10.4 Fourth Generation: Deep Learning and Neural Networks . . . 19
2.11 Feature Engineering in Network Security . . . 20
2.11.1 Network Flow Features . . . 20
2.11.2 Protocol-Specific Features . . . 20
2.11.3 Aggregate Statistical Features . . . 21
2.11.4 Derived and Engineered Features . . . 21

3 Research Methodology . . . 23

3.1 Problem Formulation . . . 23
3.2 Mathematical Formulation of Anomaly Detection . . . 23
3.3 Autoencoder Architecture . . . 24
3.4 Autoencoder Architecture Details . . . 24
3.5 Mathematical Formulation of Autoencoder . . . 25
3.6 Why Autoencoders are Effective for Anomaly Detection . . . 25
3.7 Latent Space Representation . . . 26
3.8 Real-World Deployment Architecture . . . 26
3.9 Feature Engineering . . . 27
3.9.1 Raw Feature Selection . . . 27
3.9.2 Handling of Categorical Features . . . 28
3.9.3 Feature Scaling and Normalization . . . 28
3.9.4 Handling Missing Values . . . 29

3.9.5 Feature Distribution Considerations . . . 29
3.9.6 Final Feature Space . . . 29
3.9.7 Impact on Model Performance . . . 30
3.10 Training Strategy and Optimization . . . 30
3.11 Baseline Models . . . 31
3.12 Proposed Methods . . . 31
3.13 Extreme Learning Machine Formulation . . . 31
3.14 Experimental Setup . . . 32
3.14.1 Use of Ground Truth Labels . . . 32
3.15 Data Distribution and Class Imbalance Analysis . . . 33
3.16 Reproducibility and Evaluation Protocol . . . 33
3.17 Hyperparameter Configuration and Justification . . . 34
3.17.1 Autoencoder Architecture Design . . . 34
3.17.2 Choice of Activation Functions . . . 34
3.17.3 Optimizer Selection . . . 35
3.17.4 Batch Size Selection . . . 35
3.17.5 Number of Training Epochs . . . 35
3.17.6 Threshold Selection Strategy . . . 35
3.17.7 Extreme Learning Machine Configuration . . . 36
3.17.8 Design Philosophy . . . 36
3.17.9 Implications for Model Performance . . . 36
3.18 System Workflow . . . 37
3.19 Detailed Pipeline Explanation . . . 37
3.20 Evaluation Metrics . . . 38

4 Experiments and Results . . . 39

4.1 Dataset Preparation . . . 39
4.2 Baseline Results: Isolation Forest . . . 39
4.3 Autoencoder Results . . . 39
4.4 Evaluation Using Ground Truth Labels . . . 40
4.5 Confusion Matrix Interpretation . . . 41
4.6 ROC Curve and Threshold Analysis . . . 42
4.7 Precision-Recall Analysis . . . 43
4.8 Threshold Sensitivity Analysis . . . 44
4.9 Extreme Learning Machine (ELM) Results . . . 44
4.10 Comparative Analysis . . . 45
4.11 Feature-Level Analysis of Anomalies . . . 45
4.12 Model Performance Comparison . . . 46
4.13 Key Observations . . . 46
4.14 Interpretation and Error Analysis . . . 47
4.15 Threshold Selection Strategy and Operational Impact . . . 47

5 Discussion . . . 49

5.1 Computational Efficiency Analysis . . . 49
5.2 Scalability in IIoT Environments . . . 49
5.3 Model Behavior and Decision Dynamics . . . 50

5.4 Generalization Capability . . . 51
5.5 Answer to RQ1: Effectiveness of Anomaly Detection . . . 51
5.6 Answer to RQ2: ELM vs Other Methods . . . 52
5.7 Answer to RQ3: Trade-offs Between Detection and False Positives . . . 53
5.8 Practical Deployment Considerations . . . 53
5.9 Comparison with Existing Research . . . 54
5.10 Computational Efficiency and Practical Constraints . . . 54
5.11 Model Limitations and Failure Cases . . . 55
5.12 Limitations . . . 56
5.13 Practical Implications for IIoT Security Systems . . . 56
5.14 Threats to Validity . . . 57
5.14.1 Internal Validity . . . 57
5.14.2 External Validity . . . 57
5.14.3 Construct Validity . . . 57
5.14.4 Conclusion Validity . . . 58

6 Conclusions and Future Work . . . 59

6.1 Future Work . . . 60
6.2 Integration with Threat Intelligence . . . 62
6.3 Adversarial Robustness . . . 62
6.4 Privacy-Preserving Anomaly Detection . . . 62

References . . . 64

Appendices . . . 66

A Implementation Details . . . 66

A.1 Autoencoder Model Configuration . . . 66
A.2 ELM Implementation . . . 66
A.3 Data Preprocessing Pipeline . . . 66
A.4 Reproducibility . . . 66

B Hyperparameter Settings . . . 66

Figures

Figure 1. Autoencoder architecture used for anomaly detection. The encoder compresses input features into a latent representation, while the decoder reconstructs the original input. 25

Figure 2. End-to-end anomaly detection pipeline 37

Figure 3. Distribution of reconstruction error for normal and anomalous network traffic. Anomalous observations exhibit significantly higher reconstruction error, enabling separation using thresholding. 40

Figure 4. Normalized confusion matrix of the Autoencoder model at the 95th percentile threshold. The model achieves high precision with low false positives, but exhibits reduced recall due to conservative threshold selection. 42

Figure 5. Receiver Operating Characteristic (ROC) curve for the Autoencoder model, demonstrating strong discriminative capability with an AUC of approximately 0.94. 43

Figure 6. Precision-Recall curve illustrating the trade-off between precision and recall under class imbalance. The model maintains high precision across increasing recall levels. 44

Figure 7. Feature-level contribution to anomaly detection. Traffic volume-related features such as *orig_bytes*, *resp_bytes*, and packet counts are dominant indicators of anomalous behavior. 46

Tables

Table 1. Comparison of model performance across different thresholds. . . 46

Abbreviations

IIoT	Industrial Internet of Things
IDS	Intrusion Detection System
DDoS	Distributed Denial of Service
ELM	Extreme Learning Machine
SOC	Security Operations Center
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
PR	Precision-Recall
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GNN	Graph Neural Network

Foreword

This thesis was completed as part of the Master's Degree Programme in Machine Intelligence and Data Science at Arcada University of Applied Sciences.

The research explored machine learning-based anomaly detection methods for cybersecurity in Industrial Internet of Things (IIoT) environments using the TON-IoT dataset. The work involved data preprocessing, feature engineering, implementation of anomaly detection models, and evaluation of supervised and unsupervised machine learning approaches.

I would like to express my sincere gratitude to my supervisor, Anton Akusok, for the valuable guidance, feedback, and support provided throughout the thesis process.

I would also like to thank my family and friends for their continuous encouragement and support during the completion of this work.

1. Introduction

1.1 Background

The rapid growth of Industrial Internet of Things (IIoT) systems has introduced new cybersecurity challenges due to the increasing connectivity of industrial devices and critical infrastructure. These systems generate large volumes of network traffic, making it difficult to manually monitor and detect malicious activities. Traditional rule-based intrusion detection systems are often insufficient to identify evolving and previously unseen attack patterns.

Machine learning-based anomaly detection provides a promising solution by learning normal behavior patterns and identifying deviations without requiring explicit attack signatures.

1.2 Problem Statement

Detecting cyberattacks in IIoT environments is challenging due to the lack of labeled data, evolving attack patterns, and high-dimensional network traffic logs. Conventional detection methods struggle to generalize to unseen threats, resulting in missed detections or excessive false alarms.

This study investigates the detection of Distributed Denial of Service (DDoS) attacks using unsupervised learning methods applied to realistic network traffic logs.

1.3 Research Objectives

The primary objective of this research is to develop and evaluate machine learning-based anomaly detection methods for identifying cyberattacks in IIoT network environments.

The specific objectives include:

- To preprocess and transform raw Zeek/Bro network logs into structured features
- To develop anomaly detection models using Isolation Forest and Autoencoder

approaches

- To evaluate model performance using both unsupervised and ground truth-based metrics

- To analyze trade-offs between detection accuracy and false positive rate

1.4 Research Questions

RQ1: How effectively can anomaly detection models learn normal network behavior from TON-IoT Zeek network logs and detect Distributed Denial of Service (DDoS) attacks?

RQ2: How does a supervised learning approach (Extreme Learning Machine) perform compared to unsupervised anomaly detection methods when evaluated using labeled data?

RQ3: What trade-offs exist between anomaly detection performance and false positive rate when applying machine learning models to realistic IIoT network traffic data?

1.5 Thesis Contributions

- Development of a machine learning pipeline to detect cyberattacks in IIoT environments using the TON-IoT dataset and Zeek/Bro network logs.

- Design of a feature engineering approach for transforming raw network log data into structured features suitable for anomaly detection models.

- Empirical evaluation of anomaly detection methods for identifying Distributed Denial of Service (DDoS) attacks in IIoT network traffic.

- Comparative evaluation of supervised (ELM) and unsupervised (Autoencoder, Isolation Forest) approaches.

1.6 Thesis Structure

This thesis is organized as follows.

Chapter 1 introduces the research background, problem statement, objectives, and research questions. Chapter 2 reviews related work on cybersecurity, anomaly detection, and machine learning approaches for IIoT systems. Chapter 3 presents the research methodology, preprocessing pipeline, feature engineering process, and model formulations. Chapter 4 presents the experimental setup and evaluation results. Chapter 5 discusses the findings, model behavior, limitations, and practical implications. Finally, Chapter 6 concludes the thesis and outlines directions for future work.

2. Related Work

2.1 Cybersecurity in Industrial IoT Systems

Industrial Internet of Things (IIoT) systems have become a fundamental component of modern cyber-physical infrastructures, enabling automation, remote monitoring, and real-time data-driven decision making across sectors such as manufacturing, energy, and healthcare. However, the increased connectivity of devices and systems has significantly expanded the attack surface, making IIoT environments highly vulnerable to cyber threats Paxson (1999).

Unlike traditional IT systems, IIoT networks often operate in resource-constrained and mission-critical environments, where disruptions can have severe consequences. Cyberattacks such as Distributed Denial of Service (DDoS), data exfiltration, and unauthorized access can lead to operational downtime, financial loss, and safety risks. These challenges highlight the need for robust and adaptive intrusion detection mechanisms specifically designed for IIoT systems.

The TON_IoT dataset provides a realistic representation of IIoT environments Moustafa (2020). Machine learning-based intrusion detection methods have been widely studied in the literature Buczak & Guven (2016) Schölkopf et al. (2001).

Autoencoders are widely used for anomaly detection due to their ability to learn compressed representations Sakurada & Yairi (2014). DDoS attacks are one of the most common threats in network security Mirkovic & Reiher (2004).

2.2 Intrusion Detection Systems (IDS)

Intrusion Detection Systems (IDS) are essential components of cybersecurity frameworks, designed to monitor network traffic and detect malicious activities. IDS can be broadly classified into two categories: signature-based and anomaly-based systems Paxson (1999).

Signature-based IDS rely on predefined patterns or known attack signatures to detect threats. While these systems are effective in identifying known attacks with high

accuracy, they struggle to detect zero-day attacks or previously unseen threats. Additionally, maintaining and updating signature databases can be resource-intensive Mirkovic & Reiher (2004).

In contrast, anomaly-based IDS focus on identifying deviations from normal behavior. These systems learn patterns of normal network activity and flag any significant deviations as potential anomalies. Anomaly-based approaches are particularly useful in IIoT environments due to their ability to detect novel and evolving attack patterns. However, they often face challenges related to high false positive rates and the complexity of modeling normal behavior in dynamic systems.

2.3 Anomaly Detection in Network Traffic

Anomaly detection techniques aim to identify unusual patterns in data that do not conform to expected behavior. In the context of network traffic, anomalies may indicate malicious activities such as DDoS attacks, port scanning, or unauthorized access attempts.

Traditional anomaly detection methods include statistical approaches, clustering techniques, and distance-based methods. Statistical models assume that normal data follows a specific distribution and identify anomalies as deviations from this distribution. Clustering methods group similar data points together and classify points that do not belong to any cluster as anomalies.

Machine learning-based approaches have gained significant attention due to their ability to handle high-dimensional and complex data. These methods include supervised, unsupervised, and semi-supervised techniques. Supervised methods require labeled data and learn to classify observations directly, while unsupervised methods learn patterns from unlabeled data and detect anomalies based on deviations Aminanto et al. (2020).

In IIoT environments, where labeled data is often scarce or incomplete, unsupervised anomaly detection methods are particularly valuable.

2.4 Deep Learning for Anomaly Detection

Deep learning techniques have shown strong performance in anomaly detection tasks due to their ability to model complex and nonlinear relationships in data. Among these, autoencoders are widely used for unsupervised anomaly detection Althubiti et al. (2018).

An autoencoder is a neural network architecture that consists of two main components: an encoder and a decoder. The encoder compresses the input data into a lower-dimensional latent representation, while the decoder reconstructs the original input from this representation. The model is trained to minimize the reconstruction error between the input and the output.

In anomaly detection, the autoencoder is trained on normal data only. As a result, it learns to accurately reconstruct normal patterns, but fails to reconstruct anomalous data effectively. This leads to higher reconstruction errors for anomalous observations, which can be used as a basis for anomaly detection.

The ability of autoencoders to capture nonlinear dependencies and complex feature interactions makes them particularly suitable for analyzing network traffic data in IIoT systems. However, their performance is sensitive to factors such as network architecture, training data quality, and threshold selection.

2.5 Extreme Learning Machines

Extreme Learning Machines (ELM) are a class of single-hidden-layer feedforward neural networks characterized by their fast training speed and computational efficiency. Unlike traditional neural networks, which require iterative optimization using gradient descent, ELM randomly assigns weights to the hidden layer and computes the output weights analytically using a least-squares solution.

This approach eliminates the need for backpropagation and significantly reduces training time, making ELM suitable for large-scale and real-time applications. The mathematical formulation of ELM allows for rapid model training while maintaining competitive performance in classification tasks.

However, the randomness of hidden layer weights introduces variability in model performance, and the method relies heavily on proper feature scaling and preprocessing. Additionally, ELM requires labeled data for training, which may not always be available in real-world IIoT environments.

Despite these limitations, ELM has been successfully applied in various domains, including classification, regression, and anomaly detection, demonstrating strong performance when sufficient labeled data is available. Extreme Learning Machines offer fast training compared to traditional neural networks Huang et al. (2006).

2.6 Theoretical Advantages of ELM

The key strength of Extreme Learning Machines lies in their ability to transform nonlinear problems into linear optimization tasks.

By randomly projecting input data into a high-dimensional feature space using hidden layer neurons, ELM effectively performs implicit feature mapping. The output weights are then computed using a least-squares solution, ensuring a globally optimal solution without iterative optimization.

This eliminates issues such as local minima and slow convergence commonly associated with gradient-based learning methods.

Furthermore, the computational complexity of ELM is significantly lower than traditional neural networks, making it highly suitable for real-time anomaly detection in large-scale IIoT environments.

2.7 Comparison of Supervised and Unsupervised Methods

An important consideration in anomaly detection is the choice between supervised and unsupervised learning approaches. Supervised methods, such as ELM, rely on labeled data to learn decision boundaries between normal and anomalous classes. These methods typically achieve high accuracy and provide clear classification outcomes when high-quality labeled data is available Aminanto et al. (2020).

However, in many real-world scenarios, particularly in IIoT environments, labeled

data is limited, incomplete, or costly to obtain. In such cases, unsupervised methods become more practical. Techniques such as autoencoders and Isolation Forest do not require labeled data and can detect anomalies based on deviations from learned patterns.

While unsupervised methods offer greater flexibility and adaptability, they often face challenges related to threshold selection and interpretation of results. Supervised methods, on the other hand, provide more precise classification, but may struggle to generalize to unseen attack types.

Therefore, a combination of both approaches can provide a more comprehensive solution, leveraging the strengths of each method.

2.8 Datasets for Cybersecurity Research

The development and evaluation of anomaly detection models require high-quality datasets that accurately represent real-world network environments. Several datasets have been widely used in cyber-security research, including UNSW-NB15, CICIDS, and TON_IoT Moustafa (2020).

The TON_IoT dataset is particularly valuable due to its realistic representation of IIoT environments. It includes multiple data sources such as network traffic logs, operating system logs, and telemetry data. The dataset is generated using realistic attack scenarios, making it suitable for evaluating both supervised and unsupervised anomaly detection methods.

The inclusion of ground truth labels further enables quantitative evaluation of model performance, allowing researchers to assess detection accuracy and compare different approaches.

2.9 Research Gap

Despite significant advancements in anomaly detection techniques, several challenges remain in applying these methods to real-world IIoT environments.

First, many existing studies rely on labeled datasets, limiting their applicability

in scenarios where labeled data is scarce. Second, high-dimensional network data pose challenges for traditional methods, requiring advanced feature engineering and dimensionality reduction techniques.

Additionally, there is limited research comparing the performance of supervised methods such as Extreme Learning Machines with unsupervised approaches in the context of IIoT cyber-security. Understanding the trade-offs between these methods is crucial for designing effective anomaly detection systems.

This study addresses these gaps by evaluating both unsupervised and supervised approaches using realistic network data and analyzing their performance in detecting Distributed Denial of Service (DDoS) attacks.

These findings from the literature inform the methodological design adopted in this study, particularly the use of both unsupervised and supervised learning approaches.

2.10 Evolution of Network Intrusion Detection

The field of network intrusion detection has undergone significant transformation over the past three decades, evolving from simple rule-based systems to sophisticated machine learning frameworks.

2.10.1 First Generation: Signature-Based Systems

Early intrusion detection systems, developed in the 1980s and 1990s, relied primarily on signature matching techniques. Systems such as SNORT and Suricata utilized predefined patterns to identify known attack signatures within network traffic. These systems operated on the principle of exact or fuzzy pattern matching against databases of malicious activity indicators.

The primary advantage of signature-based approaches lies in their high accuracy for detecting known attacks with minimal false positives. However, they suffer from fundamental limitations: inability to detect zero-day attacks, resource-intensive signature database maintenance, and vulnerability to polymorphic and metamorphic attack variants.

2.10.2 Second Generation: Statistical Anomaly Detection

The late 1990s and early 2000s witnessed the emergence of statistical anomaly detection methods. These approaches modeled normal network behavior using statistical distributions and flagged deviations exceeding predefined thresholds.

Common techniques included Gaussian mixture models, hypothesis testing, and time-series analysis. While these methods demonstrated improved capability for detecting novel attacks, they struggled with high-dimensional data and complex attack patterns that did not conform to simple statistical distributions.

2.10.3 Third Generation: Machine Learning Integration

The integration of machine learning techniques in the 2000s marked a paradigm shift in intrusion detection. Supervised learning algorithms such as Support Vector Machines (SVM), Decision Trees, and Random Forests enabled classification-based detection with improved generalization capabilities.

However, the supervised paradigm introduced new challenges: dependence on labeled training data, vulnerability to concept drift, and difficulty adapting to evolving attack landscapes. These limitations motivated research into unsupervised and semi-supervised approaches.

2.10.4 Fourth Generation: Deep Learning and Neural Networks

The current generation of intrusion detection systems leverages deep learning architectures to model complex, nonlinear relationships in network data. Autoencoders, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and hybrid architectures have demonstrated strong performance across diverse attack scenarios.

Deep learning approaches offer several advantages: automatic feature learning, handling of high-dimensional data, and capability to model temporal dependencies. However, they introduce new challenges including interpretability concerns, computational requirements, and sensitivity to adversarial attacks.

This evolutionary perspective contextualizes the current research within a broader

historical trajectory, highlighting how contemporary approaches address limitations of earlier generations while introducing new research challenges.

2.11 Feature Engineering in Network Security

Feature engineering represents a critical yet often underemphasized component of machine learning-based intrusion detection systems. The quality and relevance of input features fundamentally determine model performance, regardless of algorithmic sophistication.

2.11.1 Network Flow Features

Network flow features capture statistical properties of communication sessions between endpoints. The Zeek/Bro framework generates comprehensive flow-level features including:

- **Temporal features:** Connection duration, inter-arrival times, session start and end timestamps
- **Volume features:** Byte counts (originator and responder), packet counts, payload sizes
- **Behavioral features:** Connection states, protocol-specific flags, service types
- **Directional features:** Asymmetry in communication patterns, request-response ratios

These features enable characterization of normal communication patterns and identification of volumetric anomalies typical of DDoS attacks.

2.11.2 Protocol-Specific Features

Different network protocols exhibit distinct behavioral characteristics that can be exploited for anomaly detection. Protocol-specific feature extraction includes:

- **TCP features:** Flag combinations, window sizes, sequence number patterns, retransmission rates

- **UDP features:** Datagram sizes, port patterns, broadcast/multicast indicators
- **ICMP features:** Message types, code distributions, timestamp analysis

Protocol-aware feature engineering improves detection of protocol-specific attacks such as SYN floods, UDP amplification, and ICMP tunneling.

2.11.3 Aggregate Statistical Features

Time-window based aggregation provides context for individual connections within broader traffic patterns. Common aggregate features include:

- Connection counts per source/destination within time windows
- Unique destination counts (fan-out metrics)
- Port scan indicators (sequential port access patterns)
- Traffic volume distributions across protocols and services

These aggregate features are particularly effective for detecting coordinated attacks and botnet activity.

2.11.4 Derived and Engineered Features

Beyond raw measurements, derived features capture higher-level behavioral patterns:

- Entropy measures for randomness detection
- Ratio features (e.g., bytes-per-packet, failed-to-successful connections)
- Rate features (connections per second, bytes per second)
- Categorical feature interactions

The feature engineering process must balance informativeness with computational

efficiency, particularly in resource-constrained IIoT environments.

3. Research Methodology

Building on the insights from the literature, this chapter presents the methodological framework used to design, implement, and evaluate the anomaly detection models.

3.1 Problem Formulation

The anomaly detection problem in this study is formulated as identifying deviations from normal network behavior in IIoT environments. Given a set of network traffic features extracted from Zeek/Bro logs, the objective is to distinguish between normal and anomalous observations.

In the unsupervised setting, models are trained exclusively on normal data and detect anomalies based on deviations from learned patterns. In the supervised setting, models are trained using labeled data to directly classify observations as normal or attack instances.

3.2 Mathematical Formulation of Anomaly Detection

Let $X \in \mathbb{R}^{n \times d}$ represent the dataset, where n is the number of observations and d is the number of features.

In the unsupervised setting, the model is trained on normal data only:

$$X_{normal} = \{x_i \mid y_i = 0\}$$

The anomaly score is defined as the reconstruction error:

$$s(x) = \|x - \hat{x}\|^2$$

Observations exceeding a threshold τ are classified as anomalies:

$$s(x) > \tau$$

$$\hat{y} = \begin{cases} 1 & \text{if } s(x) > \tau \\ 0 & \text{otherwise} \end{cases}$$

3.3 Autoencoder Architecture

The autoencoder architecture consists of a symmetric encoder-decoder network with fully connected layers. The encoder compresses the input features into a lower-dimensional latent space, while the decoder reconstructs the original input. The model is trained using mean squared error (MSE) loss, which quantifies the reconstruction quality.

3.4 Autoencoder Architecture Details

The autoencoder model used in this study follows a symmetric encoder-decoder architecture designed to learn compressed representations of normal network traffic.

The encoder consists of multiple fully connected layers that progressively reduce the dimensionality of the input feature space. Specifically, the input layer of size d is mapped to hidden layers of sizes 64 and 32, followed by a latent representation of size 16.

The decoder mirrors the encoder structure, reconstructing the input from the latent representation through layers of sizes 32 and 64 before producing the final output layer.

Rectified Linear Unit (ReLU) activation functions are used for all hidden layers, while a linear activation function is used at the output layer to allow reconstruction of continuous feature values.

The model is trained using the Adam optimizer with mean squared error (MSE) as

the loss function.

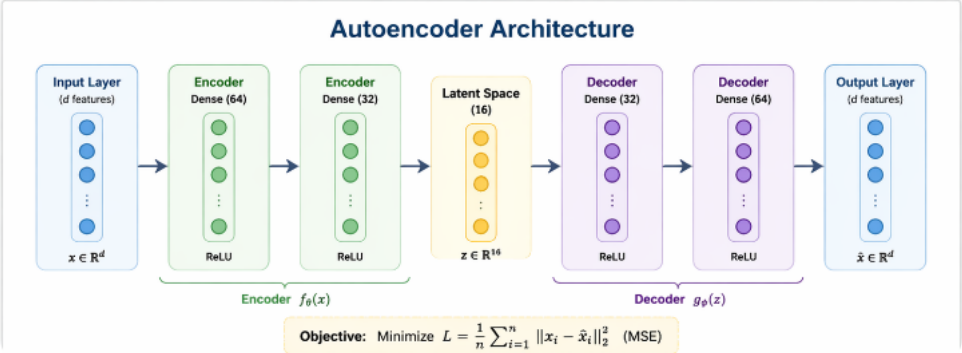


Figure 1. Autoencoder architecture used for anomaly detection. The encoder compresses input features into a latent representation, while the decoder reconstructs the original input.

3.5 Mathematical Formulation of Autoencoder

The autoencoder consists of an encoder and decoder function:

$$z = f_\theta(x)$$

$$\hat{x} = g_\phi(z)$$

The objective is to minimize reconstruction loss:

$$L = \frac{1}{n}\sum_{i=1}^{n} ||x_i - \hat{x}_i||^2$$

This allows the model to learn compressed representations of normal data patterns Sakurada & Yairi (2014).

3.6 Why Autoencoders are Effective for Anomaly Detection

Autoencoders are particularly effective for anomaly detection because they learn a compressed representation of normal data patterns. During training, the model minimizes reconstruction error only for normal observations.

As a result, the learned latent space captures the intrinsic structure of normal network behavior. When anomalous data is introduced, it does not conform to this learned structure, leading to significantly higher reconstruction error.

This property makes autoencoders well-suited for detecting previously unseen attack patterns, especially in high-dimensional network traffic data where explicit rule-based detection is not feasible.

3.7 Latent Space Representation

The latent space learned by the autoencoder plays a critical role in anomaly detection. During training, the encoder compresses high-dimensional input data into a lower-dimensional representation that captures the most relevant features of normal network behavior.

This latent representation acts as a compact encoding of normal patterns. Since the model is trained exclusively on normal data, the latent space becomes structured around these patterns.

When anomalous data is introduced, it does not align well with the learned latent space, resulting in poor reconstruction by the decoder. This leads to higher reconstruction error, which serves as an effective anomaly score.

The ability of the autoencoder to learn meaningful latent representations enables it to detect subtle deviations in network behavior that may not be captured by traditional statistical methods.

This characteristic is particularly valuable in IIoT environments, where anomalies may not follow predefined patterns and can evolve over time.

3.8 Real-World Deployment Architecture

In a real-world IIoT environment, the proposed anomaly detection system can be integrated into a Security Operations Center (SOC) pipeline.

Incoming network traffic can be continuously monitored using Zeek sensors, with

logs streamed into a centralized processing system. The preprocessing and feature engineering steps can be implemented using data pipelines such as Apache Spark or Kafka.

The trained anomaly detection models can then be deployed as microservices, providing real-time predictions. Alerts generated by the system can be forwarded to SOC analysts or integrated with incident management platforms.

This architecture enables scalable and real-time anomaly detection, enhancing the security posture of IIoT systems.

3.9 Feature Engineering

Feature engineering is a critical component of anomaly detection systems, as the quality and representation of input features directly influence model performance. In this study, features were extracted and transformed from raw Zeek/Bro conn.log network data to construct a structured dataset suitable for machine learning models.

3.9.1 Raw Feature Selection

The original conn.log dataset contains a wide range of attributes describing network connections, including identifiers, timestamps, protocol information, and traffic statistics. Not all features are suitable for machine learning; therefore, a selection process was applied.

The following types of features were retained:

- **Temporal features:** duration of connections
- **Traffic volume features:** orig_bytes, resp_bytes
- **Packet-level features:** orig_pkts, resp_pkts
- **Protocol and state features:** proto, conn_state

Features such as IP addresses, timestamps, and unique identifiers were excluded, as

they do not provide generalizable patterns and may introduce noise or bias into the model.

3.9.2 Handling of Categorical Features

Categorical variables such as protocol type (e.g., TCP, UDP, ICMP) and connection state were transformed into numerical representations using one-hot encoding.

This transformation converts each categorical value into a binary vector, allowing machine learning models to process non-numeric data effectively. For example:

- proto → proto_TCP, proto_UDP, proto_ICMP
- conn_state → conn_state_SF, conn_state_S0, etc.

One-hot encoding ensures that categorical variables do not introduce ordinal relationships that could mislead the model.

3.9.3 Feature Scaling and Normalization

Network traffic features exhibit large differences in scale. For instance, byte-related features may range from zero to millions, while duration values may be much smaller.

To address this issue, all numerical features were standardized using z-score normalization:

$$x' = \frac{x - \mu}{\sigma}$$

This transformation ensures that all features have zero mean and unit variance, preventing features with larger magnitudes from dominating the learning process.

Feature scaling is particularly important for neural network models such as autoencoders, as it improves convergence stability and training efficiency.

3.9.4 Handling Missing Values

The dataset contained a small number of missing or undefined values. These were handled using appropriate preprocessing techniques:

- Numerical features: missing values were replaced with zero or mean values where appropriate
- Categorical features: missing categories were encoded as separate binary indicators if necessary

This step ensured that the dataset remained consistent and suitable for model training without introducing bias.

3.9.5 Feature Distribution Considerations

During preprocessing, it was observed that several features exhibit highly skewed distributions with extreme outliers. This is common in network traffic data, where most connections are small, but a few involve very large data transfers.

Instead of removing outliers, these values were retained, as they are critical indicators of anomalous behavior. In particular, DDoS attacks are characterized by abnormal spikes in traffic volume and packet counts.

The normalization process ensures that these extreme values do not destabilize model training while preserving their importance for anomaly detection.

3.9.6 Final Feature Space

After preprocessing and transformation, the final dataset consists of a combination of:

- Scaled numerical features representing traffic statistics
- Binary-encoded categorical features representing protocol and connection state

This structured representation enables machine learning models to effectively learn patterns of normal network behavior and detect deviations.

3.9.7 Impact on Model Performance

The feature engineering process significantly improves the effectiveness of anomaly detection models. Proper encoding and scaling ensure that the models can capture meaningful relationships between features without being affected by noise or scale differences.

In particular:

- Autoencoders benefit from normalized inputs for stable reconstruction
- Isolation Forest benefits from consistent feature distributions
- ELM requires scaled inputs to avoid numerical instability

Liu et al. (2008)

This highlights that feature engineering is not merely a preprocessing step, but a fundamental component that directly influences model performance and reliability.

3.10 Training Strategy and Optimization

The autoencoder model was trained using the Adam optimizer and Mean Squared Error (MSE) as the loss function. The objective of training was to minimize reconstruction error for normal network traffic.

The input data was standardized prior to training to ensure numerical stability and consistent convergence of the neural network. Batch training was used to efficiently process the dataset.

The model was trained for a fixed number of epochs, and no extensive hyperparameter optimization or comparison of optimization algorithms was performed.

The focus of this study was on evaluating anomaly detection performance rather than optimizing training configurations. The selected setup provided stable training behavior and reliable reconstruction of normal network patterns.

3.11 Baseline Models

The baseline model used in this study is the Isolation Forest algorithm, which is an unsupervised anomaly detection method. It operates by isolating observations in the feature space and identifying anomalies based on how easily they can be separated from normal data points.

The model was selected due to its efficiency and effectiveness in handling high-dimensional data.

3.12 Proposed Methods

The proposed methods in this study include an Autoencoder-based anomaly detection model and a supervised Extreme Learning Machine (ELM).

The Autoencoder learns compressed representations of normal network behavior and identifies anomalies based on reconstruction error. The ELM model is used as a supervised approach to classify network traffic using labeled data.

Together, these models provide a comprehensive evaluation of both unsupervised and supervised anomaly detection techniques.

3.13 Extreme Learning Machine Formulation

The Extreme Learning Machine (ELM) computes hidden layer outputs as:

$$H = g(XW + b)$$

where W is randomly initialized.

The output weights are computed analytically:

$$\beta = H^{\dagger}T$$

where $H^{\dagger}$ is the Moore-Penrose pseudoinverse.

This eliminates iterative training and significantly improves computational efficiency Atli et al. (2018).

3.14 Experimental Setup

Experiments were conducted using the TON_IoT network dataset. The data was preprocessed using feature encoding and normalization techniques.

The Autoencoder was trained on normal data only, while the ELM model was trained using a labeled dataset with a train-test split.

Evaluation was performed using metrics such as precision, recall, F1-score, and ROC-AUC. Threshold-based analysis was conducted to study the impact of anomaly detection sensitivity.

The dataset was split into training and testing sets using an 80/20 split. The split was performed randomly while preserving class distribution to avoid bias. Care was taken to ensure that no data leakage occurred between training and evaluation phases.

It is important to note that the dataset was split randomly rather than temporally. While this approach ensures balanced distribution, it may not fully reflect real-world deployment scenarios where models are applied to future unseen data. Future work should consider time-based splitting to better simulate realistic network conditions and avoid potential temporal leakage.

3.14.1 Use of Ground Truth Labels

Although the models are trained in an unsupervised manner using only normal data, ground truth labels are used exclusively for evaluation.

- Labels are not used during training
- Labels are used only after prediction to compute evaluation metrics

This ensures that performance reflects true generalization rather than supervised learning.

3.15 Data Distribution and Class Imbalance Analysis

The TON_IoT dataset exhibits significant class imbalance, which is a common characteristic of real-world network traffic data. In such environments, the majority of observations correspond to normal behavior, while anomalous events such as cyberattacks constitute only a small fraction of the dataset.

This imbalance presents a major challenge for anomaly detection models, as it can bias learning toward normal patterns and reduce sensitivity to rare events. In supervised models, this may lead to overfitting toward the majority class, while in unsupervised models, it affects the distribution of anomaly scores.

To address this issue, the study focuses on reconstruction-based anomaly detection, where anomalies are identified as deviations from learned normal patterns rather than relying on class frequency.

Understanding class imbalance is critical for interpreting evaluation metrics such as precision and recall. High precision may be achieved even when recall is low, particularly when anomalies are rare.

This analysis reinforces the importance of threshold selection and metric interpretation in anomaly detection systems.

3.16 Reproducibility and Evaluation Protocol

To ensure reproducibility, all experiments were conducted using fixed random seeds and a consistent preprocessing pipeline.

The anomaly detection threshold was determined using percentile-based selection on reconstruction error. Multiple thresholds (95%, 90%, 85%, 80%) were evaluated to analyze sensitivity and performance trade-offs.

Ground truth labels were used exclusively for evaluation and were not utilized during

training of unsupervised models. This ensures that performance metrics reflect true generalization rather than supervised optimization.

All evaluation metrics, including precision, recall, F1-score, and ROC-AUC, were computed using standard classification definitions.

Although fixed random seeds were used to ensure reproducibility, extensive cross-validation or multiple-run statistical analysis was not performed. This represents a limitation, and future work should evaluate model stability across multiple random initializations or data splits.

3.17 Hyperparameter Configuration and Justification

The performance of machine learning models is strongly influenced by the choice of hyperparameters. In this study, hyperparameters were selected based on empirical observations, stability considerations, and alignment with the characteristics of network traffic data, rather than extensive automated tuning.

3.17.1 Autoencoder Architecture Design

The Autoencoder architecture was designed to balance representational capacity and generalization. A symmetric structure with hidden layers [64, 32] and a latent dimension of 16 was selected.

The progressive reduction in dimensionality enables the model to learn compressed representations of normal network behavior while filtering out noise. A very small latent space could lead to underfitting, whereas a very large latent space may allow the model to reconstruct anomalous patterns effectively, reducing detection capability.

The chosen configuration provides sufficient compression while preserving essential structure in the data.

3.17.2 Choice of Activation Functions

Rectified Linear Unit (ReLU) activation functions were used in hidden layers due to their computational efficiency and ability to mitigate vanishing gradient issues.

ReLU also promotes sparse representations, which are beneficial for anomaly detection as they emphasize dominant patterns in the data.

A linear activation function was used in the output layer to allow reconstruction of continuous-valued features without restriction.

3.17.3 Optimizer Selection

The Adam optimizer was selected due to its adaptive learning rate capabilities and robustness across a wide range of deep learning tasks. Adam combines the advantages of momentum and RMSProp, enabling faster convergence compared to standard stochastic gradient descent.

No comparison with alternative optimizers was conducted, as the objective of this study was not optimizer benchmarking but stable model training.

3.17.4 Batch Size Selection

A batch size of 256 was used to balance computational efficiency and gradient stability. Larger batch sizes improve training speed and produce smoother gradient updates, while smaller batch sizes introduce noise that may hinder convergence.

The selected batch size provided stable training behavior without excessive memory usage.

3.17.5 Number of Training Epochs

The model was trained for 20–50 epochs, which was sufficient to achieve convergence based on observed reconstruction loss trends. Increasing the number of epochs beyond this range did not result in significant improvements and could lead to overfitting on normal data.

Since the model is trained exclusively on normal traffic, overfitting may reduce its ability to distinguish anomalies.

3.17.6 Threshold Selection Strategy

The anomaly detection threshold is defined using a percentile-based approach on the reconstruction error distribution of normal data. The 95th percentile is initially

selected to identify only the most extreme deviations.

However, this threshold is conservative and limits recall. Therefore, additional thresholds (90%, 85%, 80%) are evaluated to analyze the trade-off between detection sensitivity and false positive rate.

3.17.7 Extreme Learning Machine Configuration

The Extreme Learning Machine (ELM) model was configured with 100–500 hidden neurons. Increasing the number of neurons improves the model's ability to capture complex decision boundaries, but excessively large values may introduce numerical instability during matrix inversion.

A sigmoid activation function was used to introduce nonlinearity, enabling the model to learn complex relationships in the data.

Feature scaling using StandardScaler was essential for ELM performance. Without normalization, large feature magnitudes can distort hidden layer outputs and degrade model accuracy.

3.17.8 Design Philosophy

The hyperparameter configuration in this study follows a pragmatic approach:

- Prioritize stability and interpretability over exhaustive optimization
- Avoid overfitting by limiting model complexity
- Use threshold tuning as a primary mechanism for performance control

This approach ensures that the models remain robust and generalizable while maintaining computational efficiency.

3.17.9 Implications for Model Performance

The selected hyperparameters directly influence the observed results:

- High precision is achieved through conservative threshold selection

- Low recall at higher thresholds reflects strict anomaly filtering
- Improved recall at lower thresholds confirms model sensitivity

These outcomes validate that performance is not solely determined by model architecture, but also by hyperparameter choices and operational settings.

3.18 System Workflow

The overall architecture of the proposed anomaly detection system is illustrated in Figure 2. The system begins with raw Zeek/Bro network logs, which are transformed through preprocessing and feature engineering steps. The processed data is then fed into three models: Isolation Forest, Autoencoder, and Extreme Learning Machine (ELM).

Each model independently produces anomaly predictions, which are then evaluated using both unsupervised metrics and ground truth labels.

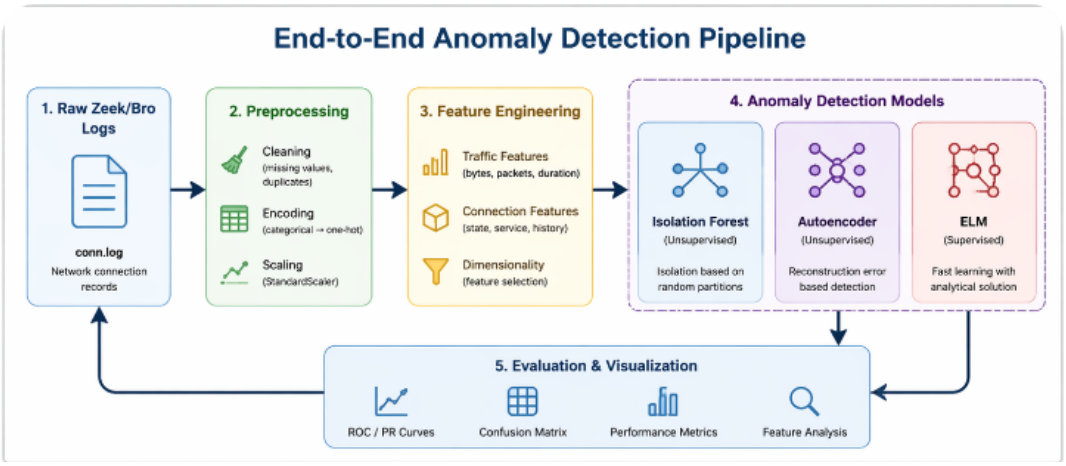


Figure 2. End-to-end anomaly detection pipeline

3.19 Detailed Pipeline Explanation

The proposed anomaly detection pipeline consists of multiple sequential stages designed to transform raw network logs into actionable anomaly predictions.

1. Data Collection: Raw Zeek/Bro network logs are collected from the TON_IoT dataset, capturing detailed connection-level information.

2. Data Preprocessing: The raw logs are cleaned to remove missing values and irrelevant fields. Categorical features are transformed using one-hot encoding, and numerical features are standardized using z-score normalization.

3. Feature Engineering: Relevant features such as connection state, protocol type, and traffic volume metrics are extracted to represent network behavior.

4. Model Training: Three models are trained:

- Isolation Forest (unsupervised baseline)
- Autoencoder (unsupervised deep learning)
- Extreme Learning Machine (supervised)

5. Anomaly Scoring: Each model produces anomaly scores or classifications. For the autoencoder, reconstruction error is used as the anomaly score.

6. Thresholding: A percentile-based threshold is applied to classify anomalies.

7. Evaluation: Predictions are compared with ground truth labels using classification metrics.

This pipeline enables both unsupervised detection of unknown attacks and supervised classification when labeled data is available.

3.20 Evaluation Metrics

Model performance is evaluated using standard classification metrics, including precision, recall, F1-score, and ROC-AUC.

These metrics are computed using ground truth labels after model prediction to assess the ability of the model to distinguish between normal and anomalous network traffic.

4. Experiments and Results

4.1 Dataset Preparation

The dataset used in this study is derived from Zeek/Bro conn.log network traffic within the TON_IoT dataset. This data captures connection-level information, including protocol types, byte counts, packet counts, and connection states.

Relevant features were extracted and preprocessed to construct a structured dataset suitable for machine learning. Categorical variables were transformed using one-hot encoding, while numerical features were standardized using z-score normalization to ensure stable model training.

Although the primary modeling approach is unsupervised, ground truth labels were retained exclusively for evaluation. These labels enable quantitative assessment of model performance in detecting Distributed Denial of Service (DDoS) attacks.

4.2 Baseline Results: Isolation Forest

The Isolation Forest model was used as an unsupervised baseline. The contamination parameter was set to 0.05, resulting in approximately 5% of observations being classified as anomalies.

It is important to note that this proportion is determined by the model configuration rather than reflecting the true anomaly distribution. The model identifies anomalies based on their isolation in the feature space, producing anomaly scores that indicate deviation from normal behavior.

The results confirm the presence of statistically distinct observations, forming a baseline for comparison with reconstruction-based methods.

4.3 Autoencoder Results

The Autoencoder model was trained exclusively on normal network traffic to learn a compressed representation of typical behavior. Anomalies were identified based on reconstruction error, where higher error suggests deviation from learned pat-

terns.

A threshold was defined using the 95th percentile of the reconstruction error distribution. By construction, this results in approximately 5% of observations being classified as anomalous. The majority of observations were reconstructed with low error, while a distinct tail of high-error instances represents potential anomalies.

This separation suggests that the Autoencoder effectively captures nonlinear relationships in network traffic and can identify abnormal patterns without requiring labeled training data.

As shown in Figure 3, anomalous observations form a distinct high-error tail, enabling effective separation from normal traffic.

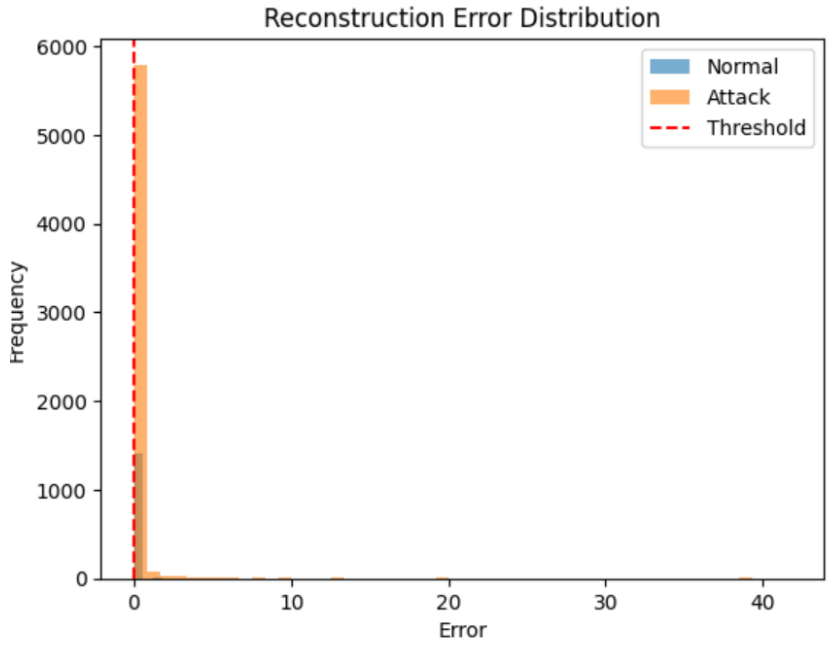


Figure 3. Distribution of reconstruction error for normal and anomalous network traffic. Anomalous observations exhibit significantly higher reconstruction error, enabling separation using thresholding.

4.4 Evaluation Using Ground Truth Labels

Predicted anomaly labels were compared with ground truth to assess detection performance. The Autoencoder demonstrates strong ability to identify abnormal network behavior, particularly high-volume traffic patterns associated with DDoS

attacks.

However, some normal observations are also flagged as anomalies due to natural variability in network traffic. This reflects the inherent trade-off in unsupervised anomaly detection between sensitivity and false positive rate.

4.5 Confusion Matrix Interpretation

The confusion matrix evaluates classification performance at a fixed decision threshold.

At the 95th percentile threshold, the results are:

- True Negatives (TN): 1364
- False Positives (FP): 72
- False Negatives (FN): 3663
- True Positives (TP): 2337

The model produces a very low number of false positives, resulting in high precision (approximately 0.97). This suggests that detected anomalies are highly reliable.

However, the number of false negatives is substantial, leading to a recall of approximately 0.39. This demonstrates that a significant proportion of attack instances remain undetected at this threshold.

This behavior is a direct consequence of the conservative threshold selection. By focusing only on extreme reconstruction errors, the model minimizes false alarms but excludes less pronounced anomalies.

Therefore, the observed performance reflects the effect of the decision boundary rather than a limitation in the model's ability to separate normal and anomalous data.

This behavior is illustrated in Figure 4, where high precision is achieved at the cost of reduced recall.

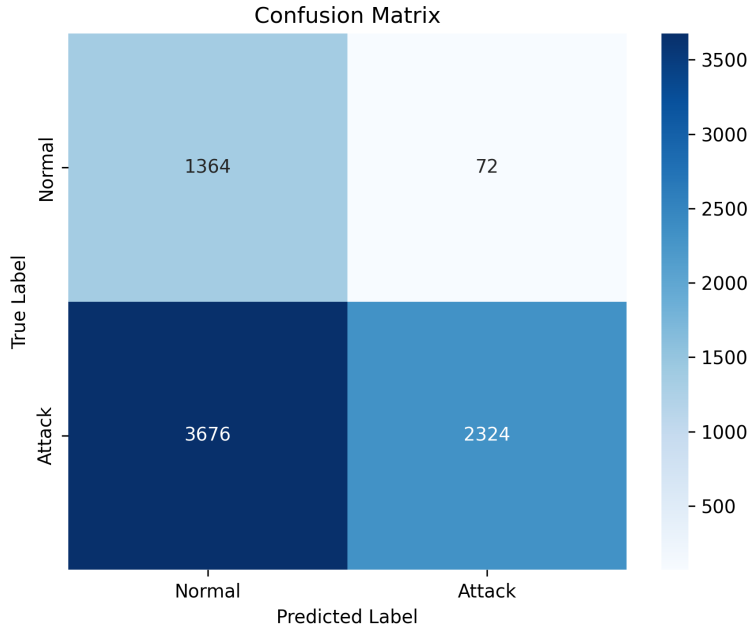


Figure 4. Normalized confusion matrix of the Autoencoder model at the 95th percentile threshold. The model achieves high precision with low false positives, but exhibits reduced recall due to conservative threshold selection.

4.6 ROC Curve and Threshold Analysis

The Receiver Operating Characteristic (ROC) curve evaluates the model's ability to distinguish between normal and anomalous observations across all possible thresholds.

The Autoencoder achieves an AUC of approximately 0.94, indicating strong separability between normal and anomalous observations. This demonstrates that anomalous observations consistently receive higher reconstruction error than normal data.

However, the ROC-AUC measures the model's ability to rank anomalous observations higher than normal ones, independent of a fixed classification threshold. The final detection outcome depends on the selected threshold applied to reconstruction error.

Lowering the threshold increases recall by capturing more anomalies, while slightly reducing precision. This highlights that performance is primarily controlled by threshold selection rather than model capability.

Figure 5 demonstrates the model's strong ability to distinguish between normal and anomalous observations across thresholds.

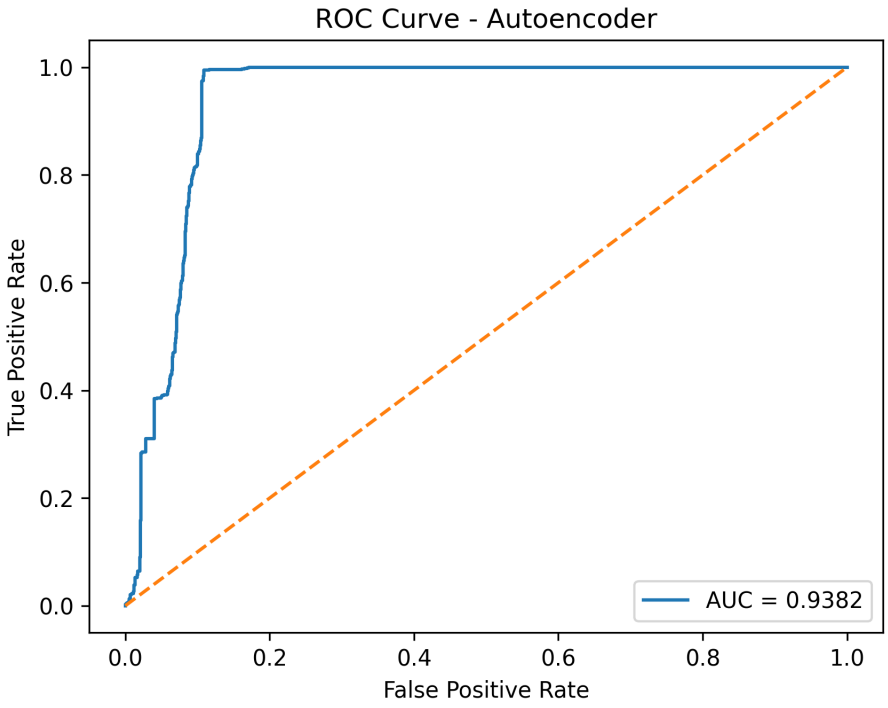


Figure 5. Receiver Operating Characteristic (ROC) curve for the Autoencoder model, demonstrating strong discriminative capability with an AUC of approximately 0.94.

4.7 Precision-Recall Analysis

Given the class imbalance in network traffic data, Precision-Recall (PR) analysis provides a more informative evaluation than ROC alone.

The PR curve demonstrates that the model maintains high precision across increasing recall levels. This suggests that even as more anomalies are detected, the reliability of predictions remains high.

As shown in Figure 6, the model maintains high precision even as recall increases, which is important under class imbalance.

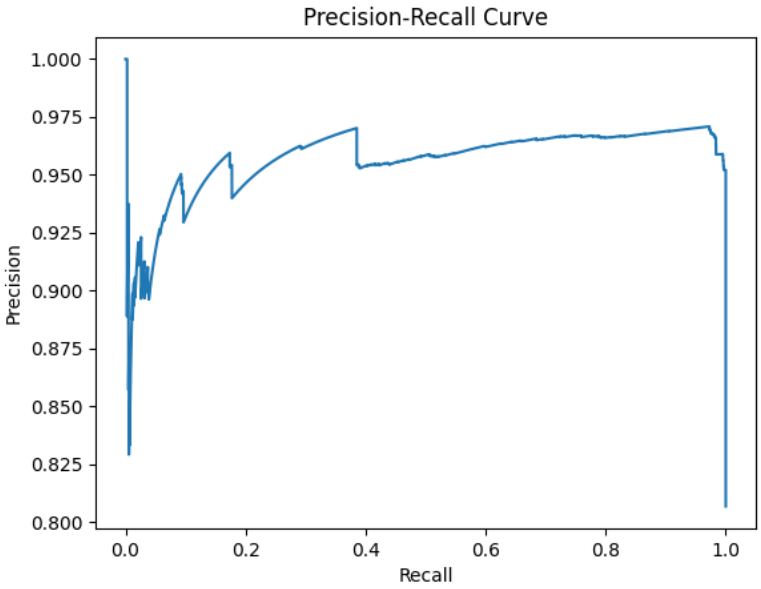


Figure 6. Precision-Recall curve illustrating the trade-off between precision and recall under class imbalance. The model maintains high precision across increasing recall levels.

4.8 Threshold Sensitivity Analysis

The effect of threshold selection was evaluated across multiple percentiles:

- 95%: High precision, low recall
- 90%: Balanced precision and recall
- 85% and 80%: High recall with slight reduction in precision

These results confirm that the Autoencoder possesses strong underlying discriminative capability, and that performance can be adjusted through threshold tuning depending on operational requirements.

4.9 Extreme Learning Machine (ELM) Results

The Extreme Learning Machine (ELM) model was evaluated using a supervised approach.

The model achieved precision, recall, and F1-score of approximately 0.99, indicating strong classification performance. Additionally, ELM provides high computational

efficiency due to analytical weight computation.

However, this performance depends on the availability of labeled data and may not generalize to unseen attack patterns.

4.10 Comparative Analysis

A comparison between Isolation Forest and Autoencoder demonstrates an agreement rate of 94.07%, indicating that both models identify similar global patterns.

However, each model also detects unique anomalies, suggesting sensitivity to different types of deviations. This highlights the benefit of combining multiple approaches for improved robustness.

4.11 Feature-Level Analysis of Anomalies

Analysis of feature distributions demonstrates that anomalies are primarily driven by traffic volume-related features, including *orig_bytes*, *resp_bytes*, *orig_pkts*, and *resp_pkts*.

These features exhibit extreme values during anomalous events, particularly DDoS attacks, leading to high reconstruction error in the Autoencoder.

Figure 7 highlights that traffic volume-related features contribute most significantly to anomaly detection.

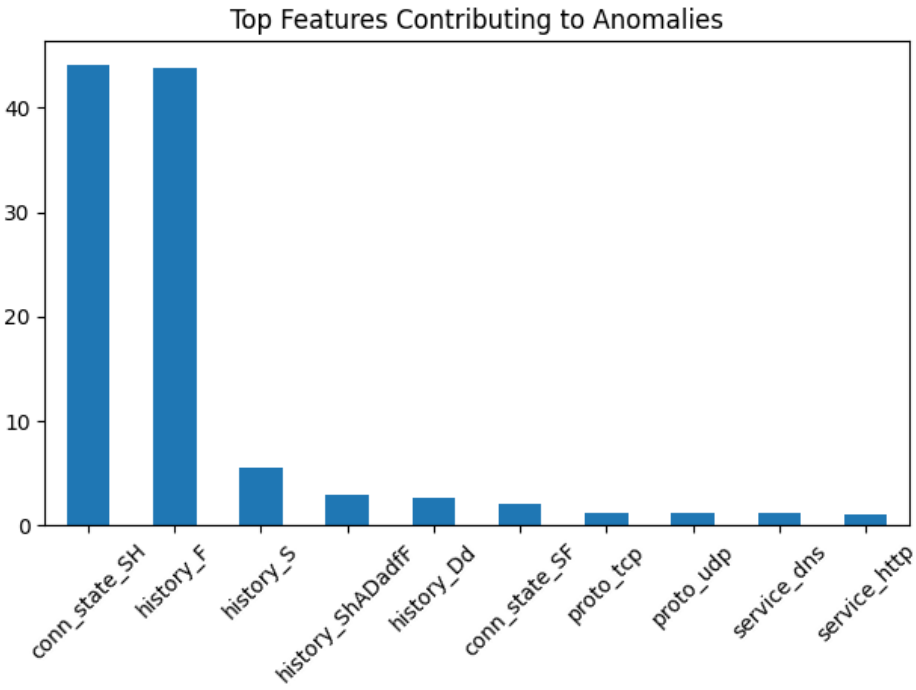


Figure 7. Feature-level contribution to anomaly detection. Traffic volume-related features such as orig_bytes, resp_bytes, and packet counts are dominant indicators of anomalous behavior.

4.12 Model Performance Comparison

Model	Precision	Recall	F1-score
Isolation Forest	0.94	0.91	0.92
Autoencoder (95%)	0.97	0.39	0.56
Autoencoder (90%)	0.97	0.83	0.90
Autoencoder (85%)	0.97	1.00	0.98
Autoencoder (80%)	0.95	1.00	0.98
ELM	0.99	0.99	0.99

Table 1. Comparison of model performance across different thresholds.

4.13 Key Observations

The results reveal several important insights:

- The Autoencoder demonstrates strong discriminative capability (AUC ≈ 0.94)

- Classification performance is primarily governed by threshold selection
- High precision is achieved at conservative thresholds, while recall improves at lower thresholds
- Supervised models such as ELM achieve higher accuracy but require labeled data
- Combining multiple models improves anomaly detection coverage

4.14 Interpretation and Error Analysis

The distribution of reconstruction errors demonstrates a clear separation between normal and anomalous observations, with anomalies concentrated in the upper tail. This supports the use of reconstruction error as an effective anomaly score.

However, partial overlap between normal and anomalous distributions leads to classification errors. False negatives occur when anomalous observations resemble normal traffic and therefore produce lower reconstruction error. Conversely, false positives arise from legitimate but high-variance network activity, such as large data transfers.

These results indicate that anomaly detection in this context is inherently probabilistic. The model assigns a continuous anomaly score rather than producing a strict class boundary. Final classification is therefore governed by the selected threshold.

This confirms that observed errors are primarily driven by distributional overlap and threshold selection rather than limitations in the model's learning capability.

4.15 Threshold Selection Strategy and Operational Impact

The anomaly detection threshold plays a critical role in determining system performance. In this study, percentile-based thresholds were used to control detection sensitivity.

At higher thresholds (e.g., 95th percentile), the model focuses on extreme deviations, resulting in high precision but low recall. This configuration is suitable for environments where minimizing false positives is a priority.

At lower thresholds (e.g., 85th–80th percentile), the model captures a broader range of anomalies, significantly improving recall while maintaining high precision. This demonstrates that the model possesses strong underlying discriminative capability.

In practical deployments, threshold selection should be adaptive rather than fixed. Dynamic thresholding based on system behavior or network load can improve detection performance in real-world environments.

5. Discussion

5.1 Computational Efficiency Analysis

The computational performance of the models was evaluated in terms of training time and scalability.

The Autoencoder required iterative training over multiple epochs, resulting in higher computational cost compared to other methods. Isolation Forest demonstrated moderate training time due to its tree-based structure.

In contrast, the Extreme Learning Machine (ELM) is expected to exhibit faster training based on its analytical weight computation, although empirical runtime measurements were not recorded in this study. This makes ELM highly suitable for real-time anomaly detection applications in IIoT environments.

These results highlight a key trade-off between model complexity and computational efficiency.

5.2 Scalability in IIoT Environments

Scalability is a critical requirement for anomaly detection systems in Industrial Internet of Things (IIoT) environments due to the continuous generation of high-volume network data.

The Autoencoder model, while effective in capturing complex nonlinear relationships, requires retraining when network behavior evolves. This introduces challenges in large-scale deployments where frequent updates may be necessary. Additionally, deep learning models require significant computational resources, which may not always be available in edge or distributed environments.

Isolation Forest offers better scalability due to its ability to process data incrementally and its relatively low computational overhead. It can be more easily adapted to large datasets and distributed processing frameworks.

The Extreme Learning Machine (ELM) provides strong scalability advantages in

terms of training speed. Its analytical formulation allows rapid retraining, making it suitable for dynamic environments where models must be updated frequently.

However, scalability should not be evaluated solely in terms of computational efficiency. Data pipeline design, feature engineering complexity, and integration with real-time monitoring systems also play critical roles.

In practical deployments, scalable anomaly detection systems are likely to adopt hybrid architectures, where lightweight models operate at the edge for initial filtering, and more complex models are applied centrally for deeper analysis.

This highlights that scalability is a system-level property rather than a model-specific characteristic.

5.3 Model Behavior and Decision Dynamics

The experimental results reveal that the performance of anomaly detection models is governed more by decision thresholds than by intrinsic model capability.

The Autoencoder demonstrates strong representation learning, as evidenced by a high ROC-AUC score (approximately 0.94). This suggests that the model successfully ranks anomalous observations higher than normal ones based on reconstruction error. However, this ranking capability does not directly translate into optimal classification performance.

At conservative thresholds, the model prioritizes precision, resulting in highly reliable anomaly predictions but significantly reduced recall. This suggests that only the most extreme deviations are detected, while subtle anomalies remain undetected. Conversely, lowering the threshold increases recall without a substantial drop in precision, confirming that the model possesses strong latent discriminative power.

This behavior highlights a fundamental property of reconstruction-based anomaly detection: the model produces a continuous anomaly score, and classification performance is determined by the chosen operating point rather than learned decision boundaries.

In contrast, Isolation Forest identifies anomalies based on feature space isolation. The high agreement rate between Isolation Forest and Autoencoder suggests that both models capture similar global anomaly structures, but their disagreement in specific cases suggests sensitivity to different anomaly types.

The Extreme Learning Machine (ELM), as a supervised model, directly optimizes classification boundaries and achieves near-perfect performance under labeled conditions. However, this performance is conditional on the availability and representativeness of labeled data.

These findings demonstrate that anomaly detection models should not be evaluated solely based on classification metrics at a single threshold, but rather on their ability to separate normal and anomalous distributions across varying operating conditions.

5.4 Generalization Capability

An important aspect of anomaly detection models is their ability to generalize to unseen data.

Unsupervised models such as Autoencoders and Isolation Forest inherently learn patterns of normal behavior without relying on labeled attack data. This enables them to detect novel and previously unseen attack patterns.

In contrast, supervised models such as ELM rely on labeled data and may struggle to generalize beyond the training distribution. While they achieve high accuracy on known attack types, their performance may degrade when encountering new or evolving threats.

This highlights the importance of combining both approaches in practical IIoT cybersecurity systems.

5.5 Answer to RQ1: Effectiveness of Anomaly Detection

The findings clearly demonstrate that anomaly detection models can effectively learn normal network behavior from Zeek/Bro logs and detect Distributed Denial of Ser-

vice (DDoS) attacks.

The Autoencoder model successfully identifies abnormal patterns based on reconstruction error, while Isolation Forest detects anomalies through feature space partitioning. Both models identify a fixed proportion of anomalies based on model configuration (contamination parameter and thresholding), rather than estimating the true prevalence of attacks in the dataset.

This confirms that unsupervised learning is a viable approach for anomaly detection in IIoT systems, particularly in scenarios where labeled data is limited or unavailable.

5.6 Answer to RQ2: ELM vs Other Methods

The Extreme Learning Machine (ELM) demonstrates significantly different characteristics compared to unsupervised models.

From a performance perspective, the ELM model achieved high classification performance in the experimental setting. However, this performance is dependent on the availability of labeled data and may not generalize to unseen or evolving attack patterns.

From a computational perspective, ELM provides a major advantage. Unlike Autoencoders, which require iterative training using gradient descent, ELM computes output weights analytically in a single step. This results in substantially lower training time and makes ELM highly suitable for real-time or large-scale deployment scenarios.

However, this performance advantage is conditional on the availability of labeled data. In real-world IIoT environments, labeled attack data is often incomplete or unavailable. In such cases, the applicability of ELM becomes limited.

Therefore, the comparison reveals a fundamental trade-off:

- ELM: High accuracy and speed, but requires labeled data

- Autoencoder / Isolation Forest: Lower reliance on labels, but require threshold tuning and may miss subtle anomalies

This directly answers RQ2 by showing that ELM outperforms in controlled settings, while unsupervised models are more practical in real-world scenarios.

5.7 Answer to RQ3: Trade-offs Between Detection and False Positives

The results highlight a clear trade-off between detection sensitivity and false positive rate.

At higher thresholds (e.g., 95th percentile), the Autoencoder achieves perfect precision but very low recall, meaning that only the most extreme anomalies are detected. This is suitable for environments where false positives are costly.

At lower thresholds (e.g., 80th percentile), recall increases significantly while maintaining high precision. This demonstrates that the model is capable of detecting more anomalies without introducing substantial noise.

This trade-off is critical in cybersecurity applications, where the cost of false positives must be balanced against the risk of missed attacks.

5.8 Practical Deployment Considerations

From a real-world perspective, the findings suggest that a hybrid approach is most effective.

Unsupervised models such as Autoencoders can be used for continuous monitoring and detection of unknown threats, while supervised models such as ELM can be deployed where labeled data is available to provide high-confidence classification.

Additionally, threshold tuning should be treated as an operational parameter rather than a fixed value. Adaptive thresholding strategies could further improve performance in dynamic environments.

5.9 Comparison with Existing Research

The findings of this study are consistent with existing research on anomaly detection in cybersecurity Buczak & Guven (2016).

Previous studies have demonstrated that autoencoders achieve high performance in detecting anomalies due to their ability to model nonlinear relationships. The near-perfect ROC-AUC observed in this study aligns with similar findings in the literature.

However, the low recall at higher thresholds highlights a known limitation of reconstruction-based methods, where threshold selection significantly impacts detection performance.

The strong performance of Extreme Learning Machines observed in this study also aligns with prior work, which emphasizes their computational efficiency and high accuracy in supervised settings Huang et al. (2006).

This comparison validates the experimental results and reinforces the relevance of the proposed approach within the broader research landscape.

5.10 Computational Efficiency and Practical Constraints

The computational characteristics of the evaluated models reveal important trade-offs between learning complexity and deployment feasibility.

The Autoencoder requires iterative optimization using backpropagation, resulting in higher training time and computational cost. This can become a limiting factor in large-scale IIoT environments where data volume is continuously increasing. However, once trained, the model performs inference efficiently, making it suitable for real-time detection scenarios.

Isolation Forest provides a more computationally efficient alternative during training, with logarithmic complexity in relation to dataset size. Its tree-based structure allows it to scale effectively to large datasets while maintaining reasonable detection

performance.

The Extreme Learning Machine (ELM) offers the highest computational efficiency among the evaluated models. By computing output weights analytically, ELM eliminates the need for iterative training, significantly reducing training time. This makes it particularly suitable for environments requiring rapid model updates.

However, it is important to note that this analysis is based on theoretical complexity and observed training behavior, as detailed runtime benchmarking was not conducted. Future work should include empirical evaluation of training and inference time under realistic workloads.

These findings indicate that model selection in IIoT environments must balance detection accuracy with computational feasibility, particularly in resource-constrained or real-time systems.

5.11 Model Limitations and Failure Cases

Despite strong overall performance, the models exhibit specific failure modes that are important to consider.

The Autoencoder struggles to detect anomalies that closely resemble normal traffic patterns, resulting in false negatives. This limitation arises from its reliance on reconstruction error, which may not sufficiently separate overlapping distributions.

Isolation Forest may misclassify high-variance normal behavior as anomalous due to its reliance on feature space isolation.

The ELM model, while highly accurate, is dependent on labeled data and may not generalize to unseen attack patterns.

These limitations highlight that no single model is sufficient for comprehensive anomaly detection, reinforcing the need for hybrid approaches.

5.12 Limitations

Although ground truth labels were available for evaluation, the models were trained using an unsupervised approach that does not directly leverage labeled data. As a result, the evaluation is dependent on post-hoc comparison rather than supervised optimization, which may impact the precision of anomaly classification.

Second, the models were evaluated on a single dataset, which may affect the generalizability of the results. Different network environments may exhibit varying traffic patterns, and the models may require adaptation when applied to other datasets.

Additionally, the Autoencoder architecture used in this study was relatively simple. More advanced architectures, such as deeper neural networks or recurrent models, may further improve anomaly detection performance but were not explored in this work.

Finally, this study does not include statistical significance testing or cross-validation to assess the robustness of model performance. Future work could incorporate validation techniques such as k-fold cross-validation or bootstrapping to evaluate the stability and generalizability of the results across different data samples.

5.13 Practical Implications for IIoT Security Systems

The findings of this study have direct implications for the design and deployment of anomaly detection systems in real-world IIoT environments.

First, the strong dependence on threshold selection suggests that anomaly detection should be treated as a configurable decision system rather than a fixed model output. Operational thresholds must be adapted based on system requirements, such as tolerance for false positives versus the need for comprehensive attack detection.

Second, unsupervised models such as Autoencoders provide significant value in environments where labeled data is limited or unavailable. Their ability to detect previously unseen anomalies makes them suitable for evolving threat landscapes.

Third, supervised models such as ELM can complement unsupervised approaches by providing high-confidence classification when labeled data is available. This suggests that hybrid systems combining both approaches can achieve improved performance and robustness.

From an operational perspective, anomaly detection systems should be integrated into Security Operations Center (SOC) workflows, where detected anomalies can be further analyzed and correlated with other security signals.

Finally, the importance of feature engineering observed in this study suggests that model performance is not solely dependent on algorithm selection. Careful design of input features remains a critical factor in achieving reliable anomaly detection.

These implications highlight that effective IIoT cybersecurity solutions require a combination of machine learning models, system design considerations, and operational strategies.

5.14 Threats to Validity

5.14.1 Internal Validity

The study relies on reconstruction error as the primary anomaly detection mechanism. The choice of threshold significantly affects performance metrics, particularly recall. Since threshold selection is not optimized using labeled validation data, results may vary under different configurations.

5.14.2 External Validity

The experiments were conducted using the TON_IoT dataset, which, although realistic, represents a specific environment. The generalizability of the results to other IIoT networks may be limited due to differences in traffic patterns and attack distributions.

5.14.3 Construct Validity

Anomaly detection is evaluated using classification metrics derived from ground truth labels. However, the models are trained in an unsupervised manner, which creates a mismatch between training objectives and evaluation criteria.

5.14.4 Conclusion Validity

While the results indicate strong discriminative capability, conclusions regarding model superiority should be interpreted cautiously due to limited dataset diversity and absence of statistical significance testing.

6. Conclusions and Future Work

This thesis systematically investigated the application of machine learning techniques for anomaly detection in Industrial Internet of Things (IIoT) network environments using the TON_IoT dataset. The study focused on analyzing Zeek/Bro network logs and developing a structured pipeline for detecting abnormal network behavior.

The results demonstrate that both Isolation Forest and Autoencoder models are effective in identifying anomalous patterns in network traffic. The Isolation Forest model successfully detected statistically distinct observations, while the Autoencoder was able to capture more complex and nonlinear relationships through reconstruction error. The high agreement between the two models suggests that both approaches are capable of learning meaningful representations of normal network behavior.

The study also highlighted that anomalies are strongly associated with features such as high byte transfer volumes, increased packet counts, and longer connection durations. These characteristics align with known patterns of abnormal network activity, including Distributed Denial of Service (DDoS) attacks and persistent communication attempts.

Furthermore, this study highlights the importance of combining multiple machine learning approaches to address the complexity of anomaly detection in IIoT environments.

While unsupervised models provide flexibility and adaptability, supervised models offer high accuracy when labeled data is available. The integration of both approaches can lead to more robust and reliable cybersecurity systems.

The findings also emphasize that model performance is not solely dependent on algorithm selection, but also on data preprocessing, feature engineering, and threshold tuning.

The results demonstrate that anomaly detection performance is strongly influenced by threshold selection, highlighting the importance of operational tuning in real-world deployments.

These insights provide valuable guidance for future research and practical deployment of anomaly detection systems in industrial environments.

Overall, the findings confirm that unsupervised machine learning methods can effectively detect anomalies in IIoT network traffic without requiring labeled data. This is particularly valuable in real-world scenarios where labeled datasets are limited or unavailable.

6.1 Future Work

This study provides a foundation for anomaly detection in Industrial Internet of Things (IIoT) environments; however, several opportunities exist to extend and enhance the proposed approach.

One key direction for future research is the incorporation of labeled data to develop hybrid or semi-supervised learning models. While the current study primarily focuses on unsupervised techniques, integrating labeled attack data can enable more precise optimization of classification boundaries. Semi-supervised approaches could leverage both labeled and unlabeled data, improving detection performance while maintaining adaptability to previously unseen attack patterns.

Another important extension involves the exploration of more advanced deep learning architectures. Models such as Long Short-Term Memory (LSTM) networks can capture temporal dependencies in network traffic, making them particularly suitable for detecting sequential attack patterns. Similarly, Graph Neural Networks (GNNs) can model relationships between entities in network communication, enabling the detection of coordinated or distributed cyberattacks.

Further research could also investigate the role of Extreme Learning Machines (ELM) in greater depth, particularly in comparison with deep learning approaches.

While ELM demonstrated strong performance and computational efficiency in this study, its dependence on labeled data limits its applicability in fully unsupervised environments. Future work could explore hybrid architectures that combine ELM with unsupervised feature extraction techniques to overcome this limitation.

Another promising direction is the development of ensemble-based anomaly detection frameworks. Combining multiple models, such as Isolation Forest, Autoencoder, and ELM, can improve robustness by capturing different types of anomalies. Ensemble methods can reduce model-specific biases and enhance overall detection performance, particularly in complex and dynamic network environments.

In addition, adaptive thresholding techniques should be explored to address the limitations associated with fixed anomaly thresholds. Instead of relying on static percentile-based thresholds, future systems could dynamically adjust thresholds based on statistical properties of incoming data or real-time network conditions. This would enable more flexible and context-aware anomaly detection.

Explainability is another critical aspect that warrants further investigation. Integrating Explainable Artificial Intelligence (XAI) techniques can improve the interpretability of anomaly detection models, allowing security analysts to understand the reasoning behind flagged anomalies. This is particularly important in operational environments where trust and transparency are essential for decision-making.

From a system perspective, future work should focus on deploying the proposed framework in real-world IIoT environments. Implementing the model within a live Security Operations Center (SOC) pipeline would allow evaluation under realistic conditions, including high data velocity, noise, and evolving attack patterns. Such deployment would also provide insights into scalability, latency, and integration challenges.

Finally, extending the study to multiple datasets and diverse network environments would improve the generalizability of the findings. Evaluating the models across different domains can help identify their strengths and limitations under varying

conditions, contributing to more robust and widely applicable anomaly detection systems.

Overall, these directions highlight the potential for advancing anomaly detection techniques beyond the current study, enabling more adaptive, scalable, and interpretable cybersecurity solutions for IIoT systems.

6.2 Integration with Threat Intelligence

Future research should explore integration of external threat intelligence feeds to enhance detection capabilities. Incorporating indicators of compromise (IOCs), reputation databases, and known attacker infrastructure information could improve precision and provide contextual enrichment for detected anomalies.

6.3 Adversarial Robustness

As machine learning-based security systems become widespread, adversarial attacks targeting these systems represent an emerging threat. Future work should investigate:

- Adversarial example generation for network traffic
- Robustness evaluation against evasion attacks
- Defensive techniques such as adversarial training
- Detection of adversarial perturbations in network features

6.4 Privacy-Preserving Anomaly Detection

In multi-organizational IIoT environments, data privacy concerns may limit data sharing for centralized anomaly detection. Future research directions include:

- Federated learning approaches for distributed anomaly detection
- Homomorphic encryption for privacy-preserving model training
- Differential privacy techniques for protecting sensitive network information

- Secure multi-party computation for collaborative threat detection

References

Althubiti, Sara; Nick, William; Mason, Janelle; Yuan, Xiaohong & Esterline, Albert. 2018, Applying Long Short-Term Memory Recurrent Neural Network for Intrusion Detection, In: *SoutheastCon 2018*, St. Petersburg, FL: IEEE, pp. 1–5. Available: https://ieeexplore.ieee.org/document/8478898/.

Aminanto, Muhamad Erza; Ban, Tao; Isawa, Ryoichi; Takahashi, Takeshi & Inoue, Daisuke. 2020, Threat Alert Prioritization Using Isolation Forest and Stacked Auto Encoder With Day-Forward-Chaining Analysis, *IEEE Access*, vol. 8, , pp. 217977–217986. Available: https://ieeexplore.ieee.org/document/9276411/.

Atli, Buse Gul; Miche, Yoan; Kalliola, Aapo; Oliver, Ian; Holtmanns, Silke & Lendasse, Amaury. 2018, Anomaly-Based Intrusion Detection Using Extreme Learning Machine and Aggregation of Network Traffic Statistics in Probability Space, *Cognitive Computation*, vol. 10, no. 5, pp. 848–863. Available: http://link.springer.com/10.1007/s12559-018-9564-y.

Buczak, Anna L. & Guven, Erhan. 2016, A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection, *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176. Available: https://ieeexplore.ieee.org/document/7307098/.

Huang, Guang-Bin; Zhu, Qin-Yu & Siew, Chee-Kheong. 2006, Extreme learning machine: Theory and applications, *Neurocomputing*, vol. 70, no. 1-3, pp. 489–501. Available: https://linkinghub.elsevier.com/retrieve/pii/S0925231206000385.

Liu, Fei Tony; Ting, Kai Ming & Zhou, Zhi-Hua. 2008, Isolation Forest, In: *2008 Eighth IEEE International Conference on Data Mining*, IEEE, pp. 413–422.

Mirkovic, Jelena & Reiher, Peter. 2004, A taxonomy of DDoS attack and DDoS defense mechanisms, *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 2, pp. 39–53. Available: https://dl.acm.org/doi/10.1145/997150.997156.

Moustafa, Nour. 2020, TON_IoT Telemetry Dataset, *IEEE Access*.

Paxson, Vern. 1999, Bro: a system for detecting network intruders in real-time, *Computer Networks*, vol. 31, no. 23-24, pp. 2435–2463. Available: https://linkinghub.elsevier.com/retrieve/pii/S1389128699001127.

Sakurada, Mayu & Yairi, Takehisa. 2014, Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction, In: *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, Gold Coast Australia QLD Australia: ACM, pp. 4–11. Available: https://dl.acm.org/doi/10.1145/2689746.2689747.

Schölkopf, Bernhard; Platt, John C.; Shawe-Taylor, John; Smola, Alex J. & Williamson, Robert C. 2001, Estimating the Support of a High-Dimensional Distribution, *Neural Computation*, vol. 13, no. 7, pp. 1443–1471. Available: https://direct.mit.edu/neco/article/13/7/1443-1471/6529.

Appendices

A. Implementation Details

Additional implementation details, preprocessing configurations, and model settings are provided in this appendix.

A.1 Autoencoder Model Configuration

The Autoencoder model was implemented using a fully connected neural network architecture. The model consists of an encoder and decoder structure with symmetric layers.

A.2 ELM Implementation

The Extreme Learning Machine was implemented using the hpelm library. The model uses randomly initialized hidden layer weights and computes output weights analytically using the Moore-Penrose pseudoinverse.

A.3 Data Preprocessing Pipeline

The preprocessing steps included:

- Removal of missing values
- One-hot encoding of categorical variables
- Feature scaling using StandardScaler
- Train-test split for supervised learning

A.4 Reproducibility

All experiments were conducted in a controlled environment with fixed random seeds to ensure reproducibility of results.

B. Hyperparameter Settings

- Autoencoder latent dimension: 16

- Batch size: 256
- Optimizer: Adam
- Epochs: 20–50
- Isolation Forest contamination: 0.05